

Intermediate (two-level logger) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/logging_practice.py` . Run it with `cd practice && python logging_practice.py` .

Used later by: the [Real-world wiring exercise](#) copies the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py` , and the [Build Task](#) writes the same function again in `practice/build_task/logging_setup.py` . A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger` .

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A logger can send its messages to more than one place at once. You can tell it: "show me only the important stuff on screen, but save everything to a file."

Each place it sends messages to is called a **handler**. You'll need two handlers — one for the screen, one for a file — and each handler gets its own cutoff level.

Here are the exact pieces to use:

- `logging.getLogger(__name__)` — get your logger.
- `logging.StreamHandler()` — a handler that prints to the screen.
- `logging.FileHandler("app.log")` — a handler that writes to a file.
- Give each handler its own level with `.setLevel(...)`.
- Attach both handlers to the logger with `.addHandler(...)`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A logger can have more than one **handler** attached, and each handler can have its own level. That's the whole trick here: one handler pointed at the screen, set to `INFO` ; one handler pointed at a file, set to `DEBUG` . The logger itself just needs to be set to the *lowest* level of any of its handlers, or messages get filtered out before they even reach the handlers.

Think of it as two filters in a row: the logger's own level is the first gate ("do I even bother with this message?"), and each handler's level is a second gate ("does *this* handler want it?"). A message has to pass both gates to end up somewhere.

Here are the exact pieces, with the calls you'll actually need:

- `logging.getLogger(__name__)` — get your logger.
- `logging.StreamHandler()` — a handler that prints to the screen.
- `logging.FileHandler("app.log")` — a handler that writes to a file.
- `handler.setLevel(logging.INFO)` / `handler.setLevel(logging.DEBUG)` — each handler's own cutoff.
- `logger.addHandler(handler)` — attach a handler (call this twice, once per handler).
- `logger.setLevel(logging.DEBUG)` — the logger's own level must be at least as permissive as its most permissive handler.
- Wrapping all of this in a function named exactly `def get_logger(name: str) -> logging.Logger:` — keep that name, the Real-world wiring exercise and the Build Task both import `get_logger` by it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what happens the *second* time this setup code runs. If `get_logger()` gets called twice — because a module got imported twice, or a function that wires up logging gets called more than once by accident — `logging.getLogger(name)` returns the *same* logger object both times, but a naive setup function will happily call `addHandler()` again, giving that logger two screen handlers and two file handlers. Every log line then prints twice, then three times, then four — a real, common bug.

That raises the actual design question: **should each module set up its own handlers, or should there be one place in the whole program that configures logging once, and every other module just calls `logging.getLogger(__name__)` to get a logger that already has handlers attached (via the root logger)?** Most real applications pick the second option — one central logging setup at startup — precisely to avoid the duplicate-handler problem, and because it's the only way to guarantee every module's logs are formatted and routed consistently.

Three more pieces, one per Advanced approach in the Solution:

- `if logger.handlers: return logger` — checked at the top of your setup function, this makes it safe to call the function more than once (per the design question above). If handlers are already attached, don't attach them again.
- `logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s")`, then `handler.setFormatter(formatter)` on each handler — turns a bare message into a full log line with a timestamp, level name, and logger name.
- `logging.config.dictConfig({...})` — configures loggers, handlers, and formatters all at once from a single dictionary (or, in a real app, a JSON/YAML file loaded into one), instead of building each handler by hand in Python code.

Difference between Basic, Intermediate, and Advanced: Basic explains handlers exist, each with its own level, and lists the exact pieces to use. Intermediate explains the two-gate filtering mechanism precisely, gives the exact calls and their order, and introduces wrapping it all in a reusable function. Advanced asks what happens when this setup code runs more than once and adds the 3 things that answer it — the guard against duplicate handlers, a real formatter, and `dictConfig` as a declarative alternative — turning "a function that sets up a logger" into one that's safe to call from anywhere, produces readable log lines, and can be reconfigured without touching code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
get a logger
set the logger's own level to the lowest one (DEBUG)

make a screen handler, set it to INFO
make a file handler pointed at "app.log", set it to DEBUG

attach both handlers to the logger

send a DEBUG message -> should show up in the file only
send an INFO message -> should show up on screen AND in the file
send a WARNING message -> should show up on screen AND in the file
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# practice/logging_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

screen_handler = logging.StreamHandler()
screen_handler.setLevel(logging.INFO)
logger.addHandler(screen_handler)

# now add a file handler the same way, set to DEBUG
```

Expected output if you run just this: nothing — a handler is attached, but no `.debug()` / `.info()` / `.warning()` call has been made yet, and the file handler is still missing. Add the file handler and the three log calls yourself, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```

define a function get_logger(name: str) -> logging.Logger:
    get a logger named after this module
    set the logger's own level to DEBUG (the lowest, most permissive)

    create a screen handler, set its level to INFO
    create a file handler pointed at "app.log", set its level to DEBUG

    attach both handlers to the logger
    return the logger

logger = get_logger(__name__)

log one DEBUG message    -> should appear in the file only
log one INFO message     -> should appear on screen AND in the file
log one WARNING message  -> should appear on screen AND in the file

```

```

# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    # add the file handler yourself, the same way, then return logger

logger = get_logger(__name__)
# add the three log calls yourself – see the Solution if stuck

```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

define a function get_logger(name: str) -> logging.Logger:
    get a logger named after this module
    if the logger already has handlers attached:
        return it as-is (don't add handlers a second time)

    set the logger's own level to DEBUG

    build a shared formatter (timestamp, level, logger name, message)

    create a screen handler, level INFO, attach the formatter
    create a file handler pointed at "app.log", level DEBUG, attach the formatter

    attach both handlers to the logger
    return the logger

logger = get_logger(__name__)
logger = get_logger(__name__) # calling it again is safe now – no duplicate han

log messages same as before, now formatted with timestamp + level + logger name

# alternative: skip building handlers by hand entirely –
# describe the same setup as one dictionary and hand it to
# logging.config.dictConfig(...), which builds the handlers for you

```

```

# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    formatter = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s")

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)

```

```
screen_handler.setFormatter(formatter)
logger.addHandler(screen_handler)

# add a file handler the same way, with the same formatter, then return logger
```

Finish the function, call `get_logger(__name__)` twice in a row to prove the guard works (only one set of handlers gets attached), then compare all 3 of your finished versions against the [Solution](#) — including its `dictConfig` version, which does the same job a completely different way.

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode is a flat script that proves the mechanism works — two handlers, two levels — run it once, it works. Intermediate wraps the same steps in a reusable, typed function with a return value. Advanced adds the guard that makes the function safe to call more than once, a shared formatter so the output is actually readable in a real log file, and a second path (`dictConfig`) that describes the whole setup declaratively instead of imperatively — which is what most production apps actually use.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)